

本物のプログラマーは **unsafe** と **型システム** を使う

拡張可能レコードを作って学ぶ黒魔術入門



Slides & Demo

konn / 2026-07-11 / 関数型まつり2026

<https://u.konn-san.com/fp2026>

自己紹介

いし い ひろ み

- 石井 大海 / konn (Twitter: @mr_konn)
 - 博士（理学）。専門は数理論理学（特に集合論）と計算機代数（Gröbner基底とか）
 - 最近はプログラミング言語理論の周辺にいる気がします
 - 宣伝：EGRAPHS '26 で **e-graph の数理モデラーへの応用**について発表したり、PLDI '26 で 松下祐介氏と **Haskell 内で Rust 流の借用システムを実現する共同研究**を発表したりしました
- 現職：株式会社 Jij ソフトウェア開発チーム（2024/11～）
 - **Rust** で**数理最適化の埋込ドメイン特化言語 (EDSL)** を開発しています
- 前職では **Haskell** で数値計算向け **EDSL** を開発

本題

本物のプログラマーは

unsafe

と

型システム

を使う

本物のプログラマーは

unsafe

「キケン」な機能への
バックドア

と

型システム

を使う

本物のプログラマーは

unsafe

「キケン」な機能への
バックドア

と

型システム

バックドアを
「安全」に包むもの

を使う

本物のプログラマーは

講演後の
皆さんのこと

unsafe

「キケン」な機能への
バックドア

と

型システム

バックドアを
「安全」に包むもの

を使う

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
 - Rust の `unsafe {}` ブロック / `unsafe fn` など
 - 所有権・借用検査をスキップして生のポインタにも触れるように
 - Haskell の `unsafePerformIO` や `unsafeCoerce` など
 - **IO 処理**をその場で（**純粋な計算として**）実行したり、ある型の値を**別の型にムリヤリ変換**したり
- 本講演では**もっぱら Haskell について**扱いますが、Rust でも思想は同じ

unsafe にまつわる迷信

😓 unsafe は「キケン」なので良くない

😄 いくら「強力な型システム」があっても **unsafe があっちゃ意味ない**じゃん

😡 **unsafe は罪！** unsafe 追放！

👻 unsafe がエルフの村を焼いたらしい

😞 unsafe はうちの裏には要らない

本当に？

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a -> b` があると任意の型の値が手に入る！（`unsafeCoerce` の **unsafe たるゆえん**）
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] -> Int` はそれ型それ自体は安全だが……
 - ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている: `sum xs = head xs + sum (tail xs)`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！（`unsafeCoerce` の **unsafe たるゆえん**）
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全 
 - ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている: `sum xs = head xs + sum (tail xs)`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！（`unsafeCoerce` の **unsafe たるゆえん**）
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全 掛けちゃだめ！
 - ① **仕様に対する誤り**： `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている： `sum xs = head xs + sum (tail xs)` 空リストで実行時エラー！

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは二つ考える
- ・ **正当性 (correctness)** : プログラムが仕様を本当に満たしているか？
 - ・ `sum :: [Int] → Int` なのに積を取っているなど
- ・ **健全性 (soundness)** : 今考えているAPIが天から与えられたとして、**「型システムなどの言語処理系が当然保証すべき安全性」**が破れないか？
 - ・ 型システムが念頭にある場合は**型安全性 (type safety)** とも言う
 - ・ 暗黙に「保証されてほしい性質」が仮定されており、文脈依存の概念
 - ・ `unsafeCoerce :: a → b` や立場によっては（厳密には）`head :: [a] → a` もそう

「不健全」の射程

unsafeCoerce も head も「キケン」！？

- ・ 型システムの健全性（型安全性とも）
 - ・ 標語：「型検査を通過したプログラムは**ヘンな振る舞い**をしない」
 - ・ ヘンな振る舞いとは？
 - ・ 最低限の要求：評価の前後で型が保たれること（保存）や、無限ループになってもいいから必ず評価が進むこと（進行）
- ・ 言語の「思想」や使い手の興味によって更に条件が増える
 - ・ Rust の所有権や Linear Haskell：「同じリソースに競合が起きない」まで保証したい！
 - ・ unsafeCoerce は明らかに不健全。なぜなら任意の値を任意の型に変換できると最悪SEGVるから
 - ・ head は尺度によって不健全。なぜなら head [] は実行時エラーになるから
- ・ ふわっと「型システムが当然保証してくれると思われる性質が破れてる」ことを不健全性と呼ぶ、と思えばよい

「unsafe は「キケン」なので良くない」

- head や tail、配列へのランダムアクセスだって「**キケン**」
- でも「そこは保証しない」というデザインチョイスになっている
 - 極力避けたいのでHaskell では非空性が保証された NonEmpty というリストの亜種を可能な限り使うことが最近では一般化しつつある
-

どう「unsafe」を使っていくべきか？

- ・使わずに済むなら使わないに越したことはない
- ・一方で、用法・用量を心得て使えば技術の幅を広げてくれる
 - ・安全性と効率性を両立したライブラリを創れるようになる
- ・どのように付き合っていく？
 - ・今回の設定：拡張可能レコード
 - ・まず**安全だが遅い実装**をしてAPI単体の健全性を確認
 - ・その後 **unsafe な処理を使った高速な実装**で差し替え
 - ・おまけ：線型型による効率と安全性の「先」

Case Study: 拡張可能レコード



Implementation

<https://u.konn-san.com/fp2026>

Case Study: 拡張可能レコード

- レコード：フィールドと値の束
 - Haskell（や Rust の struct）ではフィールドと値の組み合わせは型ごとに固定で、個別に定義する
- 拡張可能レコード（匿名レコードとも）：フィールドと型の組み合わせを自由に増減させたり、フィールドの非／存在に関する多相関数を書いたりできる
- 型安全なプラグイン機構やイフェクトシステムの実装に使える
 - 前職では拡張可能レコードをベースとした型安全なプラグインシステムを実用していました

レコード

点を表すレコード Point と円を表す Circle を定義

```
data Point = Point {x :: Double, y :: Double}

data Circle = Circle
  { x :: Double
  , y :: Double
  , radius :: Double
  }
```

拡張可能レコード例：一部のフィールドだけに興味がある

- Circle, Point の平行移動を書こうと思うと、個別に書く必要がある

OverloadedRecordDot

```
movePoint :: Double → Double → Point → Point
movePoint dx dy pt = pt {x = pt.x + dx, y = pt.y + dy}

moveCircle :: Double → Double → Circle → Circle
moveCircle dx dy c = c {x = c.x + dx, y = c.y + dy}
```

- 以下のように、特定のフィールドの有無だけ考えて書けたらなあ……

```
move :: (("x" := Double) ∈ fs, ("y" := Double) ∈ fs) ⇒
      Double → Double → Record fs → Record fs
move dx dy r = r {x = r.x + dx, y = r.y + dy}
```

あとからフィールドを増やす

どうやって実現する？

- Haskell では**型レベルに文字列・リストを持ち上げる**ことができる
- `{ x :: Double, y :: Double, name :: String }` は次のように書けそう
 - `Record '["x" := Double, "y" := Double, "name" := String]`
- 具体的にどういうインタフェースがあればいいだろう？

満たすべきAPI

型レベル文字列

```
type Record :: [(Symbol, Type)] → Type
data Record fs
```

型制約

```
type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs
```

型レベルの

lookup + fromJust

```
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
```

```
type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs
```

'(l, v)' と書くのがダルいので略記

```
insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record (l := v : fs)
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
fieldL :: ∀ f fs. ∀ l → (l ∈ fs) ⇒ Lens' (Record fs) (Lookup' l fs)
```


型族による型レベル関数（みたいなもの）

```
type Lookup :: Symbol → [(Symbol, v)] → Maybe v
type family Lookup l kvs where
  Lookup l '[] = 'Nothing
  Lookup l (l  := v ': fs) = 'Just v
  Lookup l (l' := v ': fs) = Lookup l fs

type Lookup' :: Symbol → [(Symbol, v)] → v
type Lookup' l kvs =
  FromJust
    (ShowType l :◇: Text " not found in fields " :◇: ShowType kvs)
    (Lookup l kvs)

type family FromJust e may where
  FromJust e ('Just v) = v
  FromJust e 'Nothing = TypeError e
```

このAPI、
健全？

実装して確認！

実装法：API の健全性を確かめる簡単な方法

- 効率を度外視して、「安全」な言語機能だけを使って API を実装してみる
 - undefined や unsafe 関数などは封印……
 - head とかランダムアクセスも可能な限り避ける
- これができれば、「この API を足しても少なくとも気を付けて書いた Haskell と同程度に健全」という事が確かめられる
 - 数理論理的には「定義による拡張」を試みることに対応

そんなこと出来るの？

A. GADTs[※]

使えば[”]できる！

※ Rust の GATs (Generalised Associated Types) とは異なるものです

GADTs: 一般化代数的データ型

- ・ パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

```
data V a n where
  Nil  :: V a 0
  (:>) :: a → V a n → V a (n + 1)
```

GADTs の応用例：Heterogeneous List

- 成分ごとに型がちがう Cons リストが書ける

```
data HList xs where
  HNil :: HList '[]
  (:-) :: x → HList xs → HList (x : xs)
```

```
htail :: HList (x : xs) → HList xs
htail (_ :- xs) = xs
```

```
hhead :: HList (x : xs) → x
hhead (x :- _) = x
```


実は HList が
拡張可能レコードの
実装に重要！

HList を使った実装1